

Root-Cause AI

Automated Bug Detection & Resolution

Transform application modernization with intelligent root cause analysis



ACTUAL PHOTO OF ME DURING MY FIRST WEEK AT RED HAT

My Daily Reality / The Problems I Face Daily

1. **The "Context Switch" Trap** —→ Run tests —→ Wait 20 mins —→ Test Failed
2. **Manual Forensics** Digging through hundreds of log lines to find the root cause
3. **The "Context Gathering" Loop** Logs aren't enough. I must manually hunt for code snippets to get good AI results.

Real Example: failed logs

the next three slides show the logs generated in **less than one minute**

```
1 Running global setup...
2 Installing core VSIX from /home/rwuimbmbra/Downloads/mta-vscode-extension-8.0.3-rc.1.vsix
3 Installing extensions...
4 Extension 'mta-vscode-extension-8.0.3-rc.1.vsix' was successfully installed.
5 Konveyor core extension installed/updated successfully.
6 Installing Konveyor Java VSIX from /home/rwuimbmbra/Downloads/konveyor-java-0.4.0-dev.20251112055309.dacf7d7.vsix
7 Installing extensions...
8 Extension 'konveyor-java-0.4.0-dev.20251112055309.dacf7d7.vsix' was successfully installed.
9 Java extension installed/updated successfully.
10 Waiting for VSCode to process extension dependencies...
11 Verified: redhat.java is installed (dependency of konveyor-java).
12 CleanupRepo: Successfully deleted directory /home/rwuimbmbra/Desktop/projects/editor-extensions/tests/coolstore
13 Cloning repository from https://github.com/konveyor-ecosystem/coolstore -b main
14 Code command: /usr/share/code/code --disable-workspace-trust --skip-welcome --user-data-dir=test-data-dir /home/rwuimbmbra/Desktop/projects/editor-extensions/tests/coolstore
15 VSCode opened
16 VSCode closed successfully.
17 Completed global setup.
18
19 Running 5 tests using 1 worker
20
21 [1A][2K[1/5] [solution-server-tests] [0] e2e/tests/solution-server/analysis-validation.test.ts:53:9 [0] Solution server analysis validations [0] Success rate validation tests [0] Reject solution and assert success rate
22 [1A][2K[solution-server-tests] [0] e2e/tests/solution-server/analysis-validation.test.ts:53:9 [0] Solution server analysis validations [0] Success rate validation tests [0] Reject solution and assert success rate
23 Installing core VSIX from /home/rwuimbmbra/Downloads/mta-vscode-extension-8.0.3-rc.1.vsix
24
25 [1A][2KInstalling extensions...
26
27
28 [1A][2KExtension 'mta-vscode-extension-8.0.3-rc.1.vsix' was successfully installed.
29
30
31 [1A][2KKonveyor core extension installed/updated successfully.
32
33 [1A][2KInstalling Konveyor Java VSIX from /home/rwuimbmbra/Downloads/konveyor-java-0.4.0-dev.20251112055309.dacf7d7.vsix
34
35 [1A][2KInstalling extensions...
36
37
38 [1A][2KExtension 'konveyor-java-0.4.0-dev.20251112055309.dacf7d7.vsix' was successfully installed.
39
40
41 [1A][2KJava extension installed/updated successfully.
42
43 [1A][2KWaiting for VSCode to process extension dependencies...
44
45 [1A][2KVerified: redhat.java is installed (dependency of konveyor-java).
46
47 [1A][2KcleanupRepo: Successfully deleted directory /home/rwuimbmbra/Desktop/projects/editor-extensions/tests/coolstore
48
49 [1A][2KCloning repository from https://github.com/konveyor-ecosystem/coolstore -b main
```

Real Example: cont

```
50
51 [1A][2KCode command: /usr/share/code/code --disable-workspace-trust --skip-welcome --user-data-dir=test-data-dir /home/rwuxmbra/Desktop/projects/editor-extensions/tests/coolstore
52
53 [1A][2KVSCode opened
54
55 [1A][2K(node:20166) Warning: Setting the NODE_TLS_REJECT_UNAUTHORIZED environment variable to '0' makes TLS connections and HTTPS requests insecure by disabling certificate verification.
56 (Use 'node-22 --trace-warnings ...' to show where the warning was created)
57
58 [1A][2KStarting server...
59
60 [1A][2KServer started successfully
61
62 [1A][2KWaiting for Run Analysis button to be enabled...
63
64 [1A][2KStarting analysis...
65
66 [1A][2KWaiting for analysis progress indicator...
67
68 [1A][2KAnalysis started successfully
69
70 [1A][2KVSCode closed successfully.
71
72 [1A][2K 1] [solution-server-tests] e2e/tests/solution-server/analysis-validation.test.ts:53:9 Solution server analysis validations Success rate validation tests Reject solution and assert success rate
73
74 [31m"beforeAll" hook timeout of 120000ms exceeded.[39m
75
76 | 26 | const MAX_ATTEMPTS = 15;
77 | 27 |
78 > 28 | test.beforeAll(async ({ testRepoData }) => {
79 |   |
80 | 29 | const repoInfo = testRepoData['coolstore'];
81 | 30 | vsCode = await VSCodeFactory.init(repoInfo.repoUrl, repoInfo.repoName);
82 | 31 | await vsCode.executeQuickCommand('Konveyor: Restart Solution Server');
83 |   | at /home/rwuxmbra/Desktop/projects/editor-extensions/tests/e2e/tests/solution-server/analysis-validation.test.ts:28:8
84
85 Error: [22m[31mlocator[39m[2m].[22mtoBeVisible[2m()[22m
86
87 Locator: getText('Analysis completed').first()
88 Expected: visible
89 Received: <element(s) not found>
90 Call log:
91 [2m - expect.toBeVisible with timeout 900000ms[22m
92 [2m - waiting for getText('Analysis completed').first()[22m
93
94
95 | 37 | await vsCode.startServer();
96 | 38 | await vsCode.runAnalysis();
97 > 39 | await expect(vsCode.getWindow().getText('Analysis completed').first()).toBeVisible({
98 |   |
99 | 40 | // expect.toBeVisible
```

Real Example: cont

```
87 Locator: getByText('Analysis completed').first()
88 Expected: <visible>
89 Received: <element(s) not found>
90 Call log:
91 [2m - expect.toBeVisible with timeout 90000ms [22m
92 [2m - waiting for getByText('Analysis completed').first() [22m
93
94
95 37 | await vscode.startServer();
96 38 | await vscode.runAnalysis();
97 > 39 | await expect(vscode.getWindow().getByText('Analysis completed').first()).toBeVisible({
98     |                                     ^
99 40 |     timeout: 90000,
100 41 | });
101 42 | });
102    at /home/rwurmbr/Desktop/projects/editor-extensions/tests/e2e/tests/solution-server/analysis-validation.test.ts:39:78
103
104 attachment #2: trace (application/zip)
105 test-output/solution-server-analysis-v-dccb5-ion-and-assert-success-rate-solution-server-tests/trace.zip
106 Usage:
107
108 npx playwright show-trace test-output/solution-server-analysis-v-dccb5-ion-and-assert-success-rate-solution-server-tests/trace.zip
109
110
111
112
113 [1A][2K[2/5] [solution-server-tests] e2e/tests/solution-server/analysis-validation.test.ts:59:9 Solution server analysis validations Success rate validation tests Accept solution and assert success rate
114 [1A][2K[3/5] [solution-server-tests] e2e/tests/solution-server/analysis-validation.test.ts:67:7 Solution server analysis validations Filter by "Has Success Rate" - files view
115 [1A][2K[4/5] [solution-server-tests] e2e/tests/solution-server/analysis-validation.test.ts:71:7 Solution server analysis validations Filter by "Has Success Rate" - issues view
116 [1A][2K[5/5] [solution-server-tests] e2e/tests/solution-server/analysis-validation.test.ts:75:7 Solution server analysis validations Filter by "Has Success Rate" - all incidents view
117 [1A][2K 1 failed
118 [solution-server-tests] e2e/tests/solution-server/analysis-validation.test.ts:53:9 Solution server analysis validations Success rate validation tests Reject solution and assert success rate
119 4 did not run
120
```

The Solution: Root-Cause AI Tool

AI-powered agentic log analysis + intelligent remediation

1. Run the tests and collect relevant logs

Agent executes test and isolates only the necessary log data.

2. Agent Analyze the failure and provide a hint

AI examines patterns and offer immediate clues to the root cause

3. Agent suggest solution base on the hint & code access

combines log insights with full codebase context for actionable fixes

4. Review & Apply fix

developer approves the suggested solution remotely

The Real Benefit:

Eliminates the "Context Switch" Penalty. No more waiting or manual digging. You come back to a **solution**, not a problem

Real Example: AI-Identified Root Cause

Telegram message on the user side

Bug Fix Summary

Hint:

The test's 'beforeAll' hook timed out because the 'Analysis completed' message did not become visible.

Functions to Edit:

- /home/rwurmbra/Desktop/projects/editor-extensions/tests/e2e/tests/solution-server/analysis-validation.test.ts:test.describe block

Reason:

The 'beforeAll' hook's timeout (implied to be 120s) is shorter than the 15-minute timeout explicitly set for the 'Analysis completed' expectation, causing the hook to fail prematurely.

Patch Suggestion:

Current code:

```
TypeScript
-- a/home/rwurmbra/Desktop/projects/editor-extensions/tests/e2e/tests/solution-server/analysis-validation.test.ts
@@ -15,6 +15,7 @@
import { FilterMode } from '../../enums/filter-mode.enum';

test.describe('Solution server analysis validations', () => {
  let vscode: VSCode;
  let mcpClient: MCPClient;
  let successRateBase: SuccessRateResponse;
```

Suggested fix:

```
TypeScript
++ b/home/rwurmbra/Desktop/projects/editor-extensions/tests/e2e/tests/solution-server/analysis-validation.test.ts
@@ -15,6 +15,7 @@
import { FilterMode } from '../../enums/filter-mode.enum';

test.describe('Solution server analysis validations', () => {
  test.describe.configure({ timeout: 900000 });
  let vscode: VSCode;
  let mcpClient: MCPClient;
  let successRateBase: SuccessRateResponse;
```

12:49

Rerun

Fix & Rerun

Terminate

How It Can Help Others?

The same agent that analyzes test logs can analyze logs from other products and workflows

The log analysis agent isn't tied to one specific problem. It can analyze any logs and identify root causes. The core capability is **reusable across different products** and scenarios.

Developers

Test failures, build failures, deployment issues

Support Teams

Analyze customer logs, understand problems faster

How this tool can move to production

We don't need to start from scratch

Code Analysis & Parsing

Patterns and infrastructure from code transformation work

LLM Agent Framework

Orchestration, authentication, integration patterns

Log Processing

Infrastructure for analyzing structured and unstructured logs

VSCode Integration

Extension architecture and deployment experience

Kai Team Knowledge:

People who understand code analysis and transformation. People who have built LLM agent systems. People who understand log analysis problems. People who understand how to integrate with products.

Potential Collaborators:

There may be people across Red Hat teams interested in working on this or guiding the approach. Worth exploring who's interested.

Who Benefits

Multiple groups, real improvements

Engineers & Developers

Instead of being stuck at your desk unproductive, you're free. Tests run remotely. Fixes apply remotely. You go win the ping pong tournament.

Support Teams

Analyze customer logs faster. Understand problems better. Resolve customer issues faster and more effectively. Less frustration on both sides.

Customers

When something breaks, their support experience is faster. Issues are understood quickly. Resolution is more reliable. Better experience with Red Hat products.

This Could Work

An idea worth exploring with the right people

Solves real problems for developers, support teams, and customers

Built from existing Kai knowledge and code

Can be applied across multiple Red Hat products and workflows

Improves engineer experience and customer satisfaction

How It Works: The Architecture

Proof-of-concept: developer workflow in action

The System Automatically:

- Runs your test suite
- Analyzes failures using AI agents
- Generates minimal, safe fixes
- Sends results via Telegram
- Waits for your decision

Your Three Options:

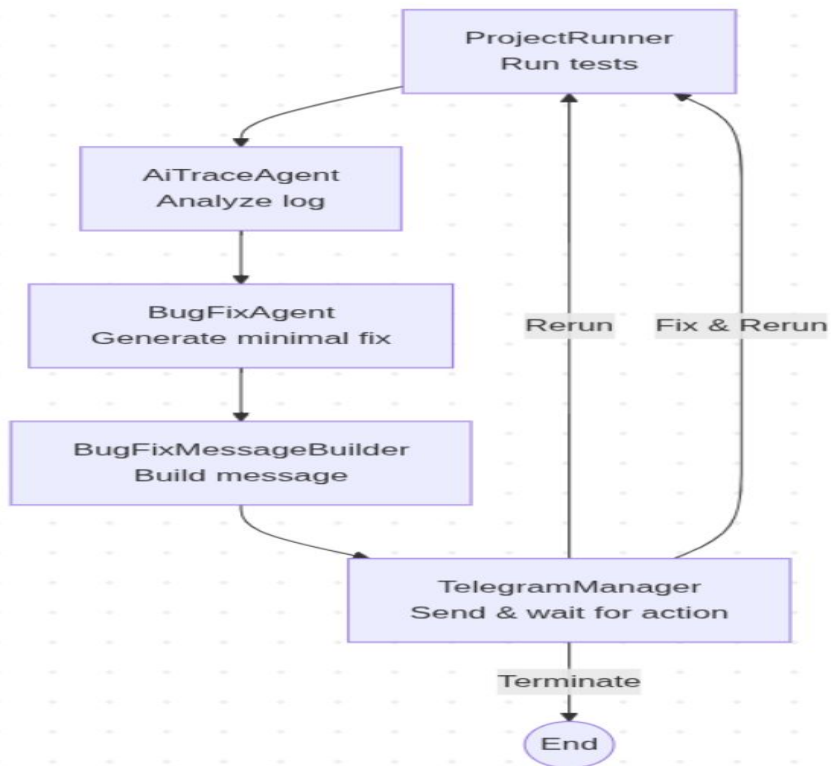
Rerun: Run tests again to get more data

Fix & Rerun: Accept the AI fix and rerun tests

Terminate: Stop and review manually

Flow Diagram

Each component has a specific responsibility



It Works

We've proven the concept. Now let's scale it.

What We've Shown:

AI can identify root causes from chaotic logs

AI can generate safe, targeted fixes

Users stay unblocked — interact via Telegram

The system works across different workflows

Future Vision: From "Tool" to "Organizational Brain"



Collaborative RAG & Organizational Memory

- Transform fixes into shared team knowledge
- Ingest runbooks, historical logs & incidents
- AI "learns" environment specifics over time



Universal Log Analysis

- Analyze ANY log (Server, CI/CD, Prod)
- Identify anomalies without format knowledge
- Zero configuration required